

THE PROMPT LADDER

One weak baseline + five controlled iterations

Name	Omer Yar Khan
Track	Backend AI Engineering
Program	FlyRank AI Internship
Assignment	Week 2 — General AI Fluency: The Prompt Ladder

1. Assignment Overview

This document shows a weak prompt improved through five controlled iterations. Each version adds exactly one named layer, and every output is compared against the version before it.

Selected task

Review one endpoint from my Task API (the Postgres-backed CRUD service I built earlier this internship) and find what's actually wrong with it, without overengineering a small intern project.

Code used for all six runs

```
@app.get("/tasks/search")
def search_tasks(title: str):
    query = f"SELECT * FROM tasks WHERE title LIKE '%{title}%'"
    cur = conn.cursor()
    cur.execute(query)
    return cur.fetchall()
```

Expected behavior

- Return tasks whose title contains the search term.
- Never let user input alter the SQL query itself.
- Return JSON shaped like the rest of the API's Task model (id, title, done) — not raw database rows.

Run 1 — Weak Baseline

Named layer: Baseline — no additional layer

PROMPT

```
Review this backend code.

@app.get("/tasks/search")
def search_tasks(title: str):
    query = f"SELECT * FROM tasks WHERE title LIKE '%{title}%'
    cur = conn.cursor()
    cur.execute(query)
    return cur.fetchall()
```

REPRESENTATIVE OUTPUT

This function fetches tasks from the database and returns them. To improve it, consider adding docstrings, type hints, unit tests, logging, caching, rate limiting, and moving to an ORM like SQLAlchemy for better maintainability. Pagination would also help for large result sets.

Comparison notes

What changed	Nothing — this is the intentionally weak baseline.
What improved	Not applicable — first output.
What still failed	Never named the SQL injection risk at all, the one bug that actually matters. Instead it suggested unrelated infrastructure (caching, an ORM, rate limiting) for a four-line function.
What I would try next	State a specific goal so the model looks for bugs and security issues instead of general "cleanup" ideas.

Run 2 — Clearer Goal

Named layer: Clearer goal

PROMPT

Review this backend code and identify functional bugs and security vulnerabilities.

```
@app.get("/tasks/search")
def search_tasks(title: str):
    query = f"SELECT * FROM tasks WHERE title LIKE '%{title}%'"
    cur = conn.cursor()
    cur.execute(query)
    return cur.fetchall()
```

REPRESENTATIVE OUTPUT

The main issue is a SQL injection vulnerability: **title** is interpolated directly into the query string with an f-string, so a value like ' **OR '1'='1** would alter the query. Use a parameterized query instead. Beyond that, the function is also missing error handling, doesn't validate that title is non-empty, isn't type-hinted, and should probably use connection pooling and an ORM for long-term maintainability.

Comparison notes

What changed	Asked specifically for bugs and security vulnerabilities instead of a general review.
What improved	Named the actual vulnerability — SQL injection via f-string interpolation — and explained why it's exploitable.
What still failed	Still tacked on unrelated production concerns (connection pooling, an ORM migration) that don't fit a small intern CRUD project.
What I would try next	Give real context about the project's size and existing patterns so recommendations stay proportionate.

Run 3 — Real Context

Named layer: Real context

PROMPT

Review this backend code and identify functional bugs and security vulnerabilities. This function is part of a small intern-built Task CRUD API using FastAPI and psycopg, following an existing repository pattern (see TaskRepository). It is not intended to introduce new architecture or dependencies — only the search endpoint itself needs fixing.

```
@app.get("/tasks/search")
def search_tasks(title: str):
    query = f"SELECT * FROM tasks WHERE title LIKE '%{title}%'"
    cur = conn.cursor()
    cur.execute(query)
    return cur.fetchall()
```

REPRESENTATIVE OUTPUT

The SQL injection is still the critical issue: build the query with a parameterized placeholder, e.g. `cur.execute("SELECT id, title, done FROM tasks WHERE title ILIKE %s", (f'#{title}%'))`. Since this project already has a repository pattern, this query belongs in `PostgresTaskRepository` as a new method (e.g. `search_by_title`), not inline in the route. No new dependencies or ORM needed for this fix.

Comparison notes

What changed	Told the model the project's actual scale and existing architecture.
What improved	Stopped recommending an ORM/connection-pool rewrite and instead pointed the fix at the existing repository pattern — exactly how the rest of this project is structured.
What still failed	The response was one dense paragraph — the fix, the reasoning, and the placement advice ran together, making it harder to scan quickly.
What I would try next	Ask for a structured, sectioned response so the pieces are easier to separate.

Run 4 — Specified Output Format

Named layer: Specified output format

PROMPT

Review this backend code and identify functional bugs and security vulnerabilities. This function is part of a small intern-built Task CRUD API using FastAPI and psycopg, following an existing repository pattern. Return the review using these sections: 1. What the code does 2. Bugs or risks 3. Recommended fix 4. Revised code.

```
@app.get("/tasks/search")
def search_tasks(title: str):
    query = f"SELECT * FROM tasks WHERE title LIKE '%{title}%'"
    cur = conn.cursor()
    cur.execute(query)
    return cur.fetchall()
```

REPRESENTATIVE OUTPUT

1. What the code does: Searches tasks by a partial title match and returns matching rows.
2. Bugs or risks: SQL injection via f-string interpolation.
3. Recommended fix: Use a parameterized ILIKE query, placed in PostgresTaskRepository.
4. Revised code:

```
def search_tasks(title: str):
    cur.execute("SELECT id, title, done FROM tasks WHERE title ILIKE %s",
                (f'%{title}%',))
    return cur.fetchall()
```

Comparison notes

What changed	Requested four explicit sections instead of free-form prose.
What improved	Easy to scan — the bug and the fix are now clearly separated instead of buried in a paragraph.
What still failed	<i>This made the output worse in one real way: the "revised code" still returns cur.fetchall() directly — raw database tuples, not the JSON shape (id, title, done) the rest of the API returns. Fixing the security bug quietly introduced a contract-breaking regression that Run 3's version didn't have.</i>
What I would try next	Add explicit verification requirements so the model checks the revised code against the existing response contract before handing it back.

Run 5 — Verification Requirements

Named layer: Verification requirements

PROMPT

Review this backend code and identify functional bugs and security vulnerabilities. This function is part of a small intern-built Task CRUD API using FastAPI and psycpg, following an existing repository pattern. Return the review using these sections: 1. What the code does 2. Bugs or risks 3. Recommended fix 4. Revised code. Before returning the revised code, verify that it returns a list of JSON-serializable objects shaped exactly like the existing Task model (id, title, done) – not raw database rows – and uses only parameterized queries.

```
@app.get("/tasks/search")
def search_tasks(title: str):
    query = f"SELECT * FROM tasks WHERE title LIKE '%{title}%'"
    cur = conn.cursor()
    cur.execute(query)
    return cur.fetchall()
```

REPRESENTATIVE OUTPUT

1. What the code does: Searches tasks by a partial title match.
2. Bugs or risks: SQL injection via f-string interpolation.
3. Recommended fix: Parameterized ILIKE query; convert rows to the Task shape before returning.
4. Revised code:

```
def search_tasks(title: str):
    cur.execute("SELECT id, title, done FROM tasks WHERE title ILIKE %s",
(f'"{title}%',))
    rows = cur.fetchall()
    return [{'id': r[0], 'title': r[1], 'done': r[2]} for r in rows]
```
5. Verification: query is parameterized (no string interpolation of title into SQL); the return shape now matches the Task model's three fields, unlike the previous version's raw tuples.

Comparison notes

What changed	Added an explicit verification step tied to the exact contract the code must preserve.
What improved	Caught and fixed the regression from Run 4 — the revised code now returns the correct shape, and the review states explicitly that it checked this before answering.
What still failed	The verification note repeats information already stated in "Recommended fix," making the answer slightly redundant.
What I would try next	Add a constraint against scope creep and repetition, and finalize the prompt into a reusable template.

Run 6 — Constraints

Named layer: Constraints

PROMPT

Review this backend code and identify functional bugs and security vulnerabilities. This function is part of a small intern-built Task CRUD API using FastAPI and psycopg, following an existing repository pattern. Return the review using these sections: 1. What the code does 2. Bugs or risks 3. Recommended fix 4. Revised code 5. Final verification. Before returning the revised code, verify that it returns objects shaped exactly like the existing Task model (id, title, done) and uses only parameterized queries. Do not introduce new dependencies, do not suggest architectural changes outside this one function, and do not repeat the same point across sections.

```
@app.get("/tasks/search")
def search_tasks(title: str):
    query = f"SELECT * FROM tasks WHERE title LIKE '%{title}%'"
    cur = conn.cursor()
    cur.execute(query)
    return cur.fetchall()
```

REPRESENTATIVE OUTPUT

1. What the code does: Searches tasks by partial title match.

2. Bugs or risks: SQL injection via f-string interpolation into the query.

3. Recommended fix: Parameterized ILIKE query, moved into PostgresTaskRepository.search_by_title().

4. Revised code:

```
def search_by_title(self, term: str) -> list[Task]:
    with pool.connection() as conn, conn.cursor() as cur:
        cur.execute("SELECT id, title, done FROM tasks WHERE title ILIKE %s", (f'{term}%',))
        rows = cur.fetchall()
        return [Task(id=r[0], title=r[1], done=r[2]) for r in rows]
```

5. Final verification: query is parameterized; no new dependencies introduced; return type matches TaskRepository's other methods exactly.

Comparison notes

What changed	Added explicit boundaries against scope creep and repeated explanations.
What improved	The response stayed tightly scoped to the one function, matched the existing repository's method style (returning Task objects through the same connection pool other methods use), and dropped the redundant restatement from Run 5.
What still failed	Nothing major for this task's scope — the remaining gap is that it still doesn't add a test for the fix, which wasn't asked for.

What I would try next	For a stranger reusing this prompt, add a line explicitly inviting them to name their own project's existing patterns — that's what made Run 3 onward actually useful here.
------------------------------	---

2. Final Reusable Prompt

Written as a template so another backend learner can use it without knowing the personal context behind this assignment.

```
Act as an experienced [LANGUAGE/Framework] backend engineer.

Review the code below and identify:
- Functional bugs
- Security vulnerabilities
- Maintainability concerns
- Improvements appropriate to the size and purpose of the project

Project context:
This code belongs to [PROJECT TYPE]. It follows [EXISTING PATTERN OR ARCHITECTURE].
Do not introduce new dependencies or architectural changes outside this code.

Return the review using these sections:
1. What the code currently does
2. Bugs or risks
3. Recommended fix
4. Revised code
5. Final verification

Before returning the revised code, verify that it:
- Uses only parameterized queries (no string interpolation of user input into SQL)
- Returns data shaped exactly like [EXISTING RESPONSE CONTRACT]
- Does not repeat the same point across sections

Code:
[PASTE CODE]
```

3. Reflection

The biggest single jump came from adding real context in Run 3 — it's what stopped the model from recommending an ORM migration for a four-line function. But context alone wasn't enough: Run 4's output format requirement made the review easier to scan while silently reintroducing a bug the earlier, messier version didn't have. That's the honest failure in this ladder, and it's the one worth remembering — a cleaner-looking answer is not automatically a more correct one. Verification in Run 5 caught it, and constraints in Run 6 kept the final version from drifting back into scope creep. Each layer bought something real, but none of them was free.

4. Submission Checklist

- ✓ Six runs total: one baseline plus five iterations
- ✓ Exactly one named layer added per iteration
- ✓ Representative output included for every run
- ✓ Four comparison notes included for every run
- ✓ At least one honest example of a change making the output worse (Run 4)

✓ Final prompt reusable by another person